SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1715**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

<u>Analytical Problem Solving</u>

Code of Conduct:

I P.L.OAVIKSHA(192572001)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P L Oaviksha

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

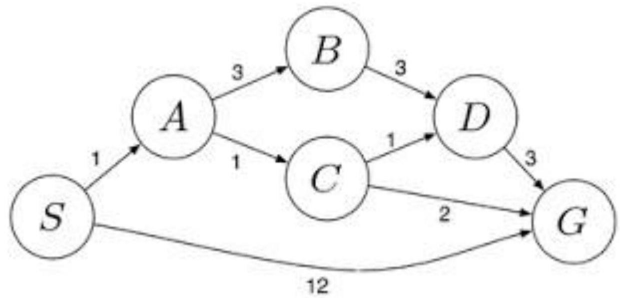
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

SIMATS ENGINEERING

SAVEETHA E OF MEDICAL AND TECHNICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO1 – ASSESSMENT TOOL 2

Constraint-Based Problem Solving

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

QUESTIONS: 3 Duration: 1 Hour **Total Marks: 30**

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total Marks (30)
Problem Understanding	25% (2.5)	2.4 /2.5	2.4 /2.5	2.4 /2.5	7.2 /7.5
Constraint Analysis	25% (2.5)	2.4 /2.5	2.4 /2.5	2.4 /2.5	7.2 /7.5
Solution Development	25% (2.5)	2.4 /2.5	2.4 /2.5	2.4 /2.5	7.2 /7.5
Application of Concepts	15% (1.5)	1.44 /1.5	1.44 /1.5	1.44 /1.5	4.32 /4.5
Justification	10% (1)	0.96 /1	0.96 /1	0.96 /1	2.88 /3
Total per Question	10 Marks	9.6 /10	9.6 /10	9.6 /10	28.8 /30

TOTAL MARK 96/100

ANNEXURE A

ANALYTICAL PROBLEM SOLVING

Question 1: Water Jug Problem

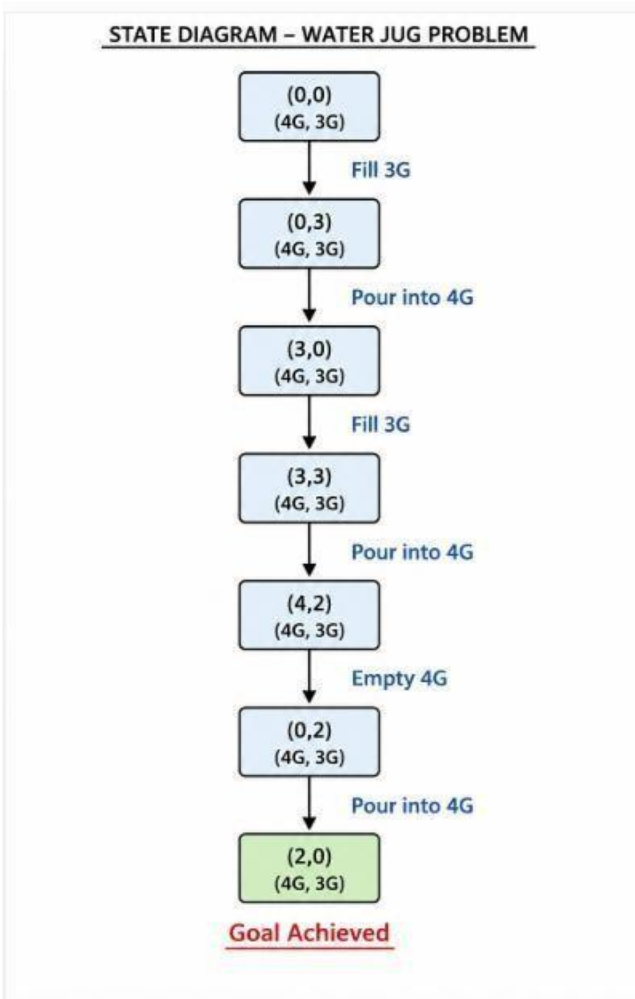
Problem Understanding:

Two jugs are available: 4-gallon and 3-gallon. The objective is to obtain exactly 2 gallons in the 4-gallon jug.

Initial State: (0,0)

Goal State: (2,x)

Step	Action	State (4G,3G)
1	Fill 3-gallon jug	(0,3)
2	Pour into 4-gallon jug	(3,0)
3	Fill 3-gallon jug again	(3,3)
4	Pour into 4-gallon until full	(4,2)
5	Empty 4-gallon jug	(0,2)
6	Pour remaining 2 gallons into 4-gallon jug	(2,0)



Algorithm:

1. Fill 3G.
2. Pour into 4G.
3. Fill 3G again.
4. Pour until 4G is full.
5. Empty 4G.
6. Pour remaining water into 4G.

Pseudocode:

START

Fill Jug3

Pour Jug3 into Jug4

Fill Jug3

Pour Jug3 into Jug4 until full

Empty Jug4

Pour remaining water into Jug4

STOP

Conclusion:

Exactly 2 gallons are obtained in the 4-gallon jug.

Question 2: Mars Rover Intelligent Agent

Aim

To analyze the Mars Rover as an intelligent agent by identifying its percepts, operating environment, actions, performance measures, and suitable agent architecture.

Problem Understanding

The Mars Rover is an autonomous robotic vehicle developed to explore the surface of Mars. It performs scientific experiments, collects rock and soil samples, captures images, avoids obstacles, and transmits valuable information back to Earth without continuous human control.

PEAS Description

PEAS Component	Description
Performance Measure	Collect maximum samples, avoid obstacles, conserve battery, transmit accurate data, complete mission successfully
Environment	Mars surface, rocks, hills, craters, dust storms, varying temperatures, sunlight
Actuators	Wheels, robotic arm, drill, camera rotation, antenna, sample collector
Sensors (Percepts)	Cameras, temperature sensor, proximity sensor, radiation sensor, battery sensor, gyroscope, spectrometer

(i) Percepts

The Mars Rover continuously receives information from its sensors.

- Camera images
- Rock characteristics

- Soil composition
- Surface temperature
- Radiation level
- Distance to obstacles
- Wheel position
- Battery level
- Tilt angle
- GPS/Navigation data (relative positioning)

(ii) Operating Environment

Property	Description
Observability	Partially Observable
Deterministic	No (Stochastic)
Episodic/Sequential	Sequential
Static/Dynamic	Dynamic
Discrete/Continuous	Continuous
Number of Agents	Single Agent

Explanation

- The rover cannot observe every part of Mars at once.
- Environmental conditions change continuously.
- Every action affects future decisions.
- The rover operates independently.

(iii) Actions Performed by the Rover

The Mars Rover can:

- Move forward
- Move backward
- Turn left
- Turn right
- Stop
- Capture images
- Drill rocks
- Collect soil samples
- Analyze samples
- Store collected samples
- Send data to Earth
- Recharge using solar energy

(iv) Performance Evaluation

The rover's performance is evaluated using the following criteria:

- Number of successful samples collected
- Accuracy of scientific analysis
- Distance safely travelled
- Obstacle avoidance efficiency
- Battery utilization
- Communication success with Earth
- Mission completion time

(v) Suitable Agent Architecture

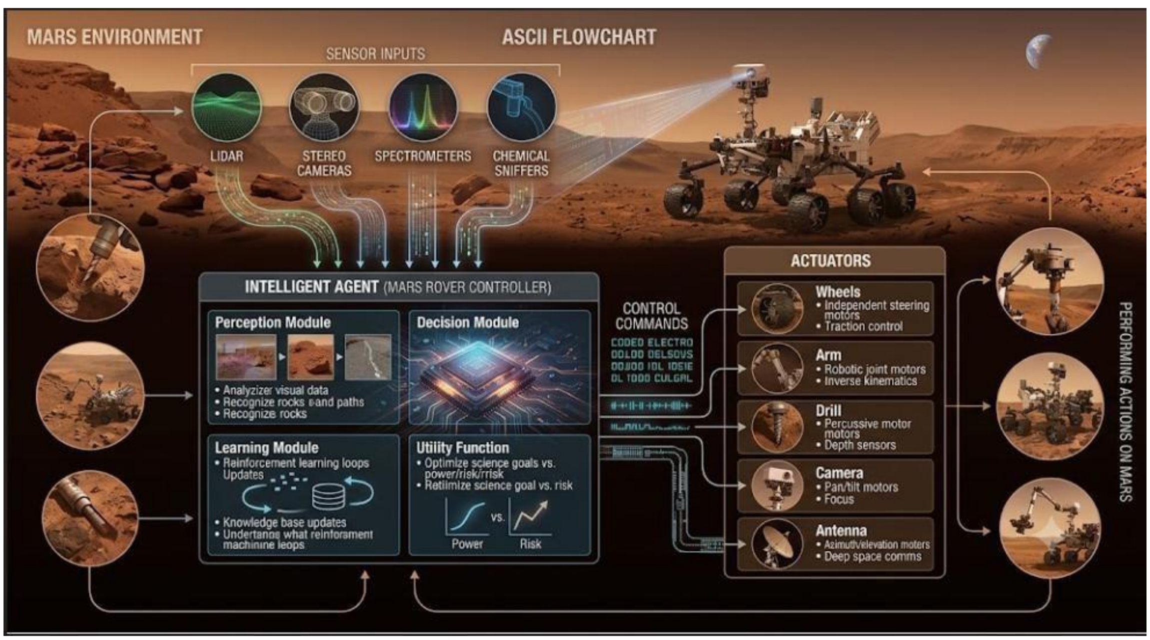
Utility-Based Learning Agent

Reason

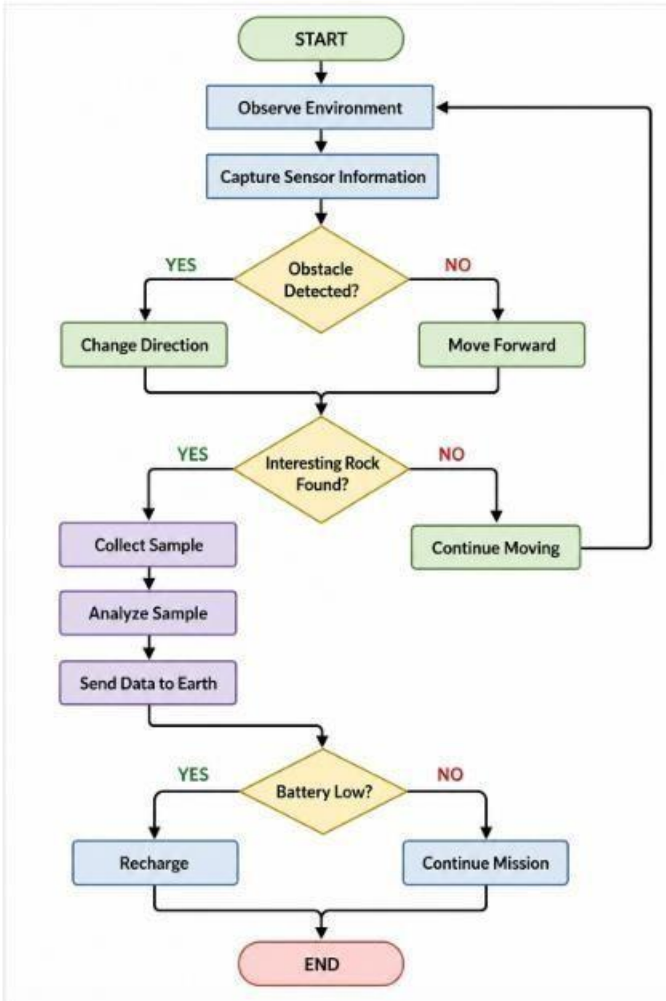
A Utility-Based Learning Agent is the most suitable because it:

- Learns from previous experiences
- Chooses the safest route
- Conserves battery power
- Maximizes scientific discoveries
- Adapts to changing environmental conditions
- Makes intelligent decisions under uncertainty

Agent Architecture Diagram



Working Flowchart



Algorithm

1. Start the rover.
2. Observe the surrounding environment.
3. Detect obstacles using sensors.
4. If an obstacle is found, change direction.
5. Otherwise, move forward.
6. Search for rocks and soil samples.
7. If a sample is detected, collect and analyze it.
8. Send the analyzed data to Earth.
9. Check the battery level.

10. Recharge using solar panels if required.
11. Repeat until the mission is completed.

Pseudocode

```
START
Initialize Rover
WHILE Mission_Not_Completed DO
    Observe Environment
    IF Obstacle_Detected THEN
        Turn Left
    ELSE
        Move Forward
    ENDIF
    IF Sample_Found THEN
        Collect Sample
        Analyze Sample
        Transmit Data
    ENDIF
    IF Battery_Level < Threshold THEN
        Recharge Battery
    ENDIF
END WHILE
STOP
```

Conclusion

The Mars Rover is an intelligent autonomous system that operates in a partially observable, dynamic, stochastic, and continuous environment**.** A Utility-Based Learning Agent is the most suitable architecture because it enables the rover to learn from experience, optimize its decisions, conserve energy, and maximize the success of scientific exploration on Mars. This approach improves mission efficiency and ensures reliable autonomous operation in the challenging Martian environment.

Question 3: 8-Queens Problem

Aim

To formulate the 8-Queens Problem as a search problem and solve it using the Backtracking Search Algorithm (Depth First Search)**.**

Problem Statement

The 8-Queens Problem is one of the most popular problems in Artificial Intelligence.

The objective is to place 8 queens on an 8 × 8 chessboard so that no queen attacks another queen**.**

A queen attacks another queen if both are in:

- The same row
- The same column
- The same diagonal

The challenge is to find an arrangement where all eight queens are placed safely.

Problem Formulation

Initial State

An empty 8 × 8 chessboard.

0 Queens placed

State

A state is a partially filled chessboard where queens are placed safely.

Example:

Column 1 → Row 1

Column 2 → Row 5

Column 3 → Row 8

Actions

- Place one queen in a safe row of the next column.
- Continue until all queens are placed.

Goal State

A valid arrangement where:

- All 8 queens are placed.
- No two queens attack each other.

Path Cost

Each queen placement has a cost of **1**.

Total Cost = **8**

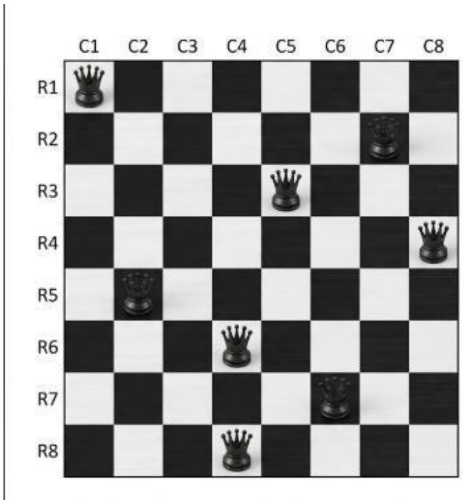
Search Strategy

- The Backtracking Search Algorithm is used.
- Backtracking is based on Depth First Search (DFS)**.**
- It places one queen at a time.
- If a conflict occurs, the algorithm removes the previous queen and tries another position.
- This process continues until a valid solution is found.

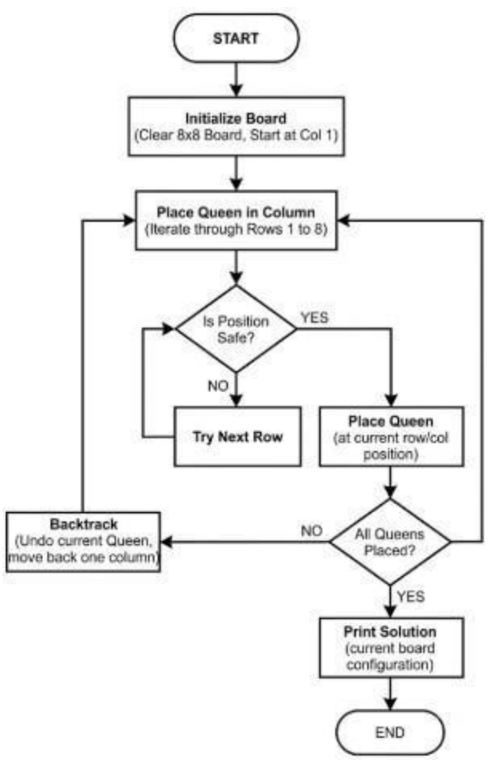
Working Steps

- Start with Column 1 and place the first queen in a safe row.
- Move to the next column and check each row for a safe position.
- Place the queen if the selected position is safe (no row or diagonal conflict).
- Repeat the process for each remaining column until all queens are placed.
- If a conflict occurs, remove the previously placed queen (Backtracking) and try the next available row.
- Continue backtracking whenever necessary until a valid arrangement is found.
- Stop when all 8 queens are placed safely without attacking each other.

Final Chessboard Solution



Flowchart



Algorithm

1. Start with the first column.
2. Place a queen in a safe row.
3. Move to the next column.
4. Check if the position is safe.
5. If safe, place the queen.
6. If unsafe, try another row.
7. If no row is safe, backtrack to the previous column.
8. Repeat until all queens are placed.

Pseudocode

```
FUNCTION Solve(Column)
IF Column == 8 THEN
    RETURN TRUE
FOR Row = 1 TO 8
    IF Safe(Row, Column) THEN
        Place Queen
        IF Solve(Column + 1) THEN
            RETURN TRUE
        ENDIF
        Remove Queen
    ENDIF
END FOR
RETURN FALSE
```

Time Complexity

- Worst Case: **O(8!)**
- Space Complexity: **O(8)** (using recursion stack)

Applications

- Artificial Intelligence
- Constraint Satisfaction Problems (CSP)
- Puzzle Solving
- Scheduling Systems
- Resource Allocation
- Robot Navigation

Advantages

- Easy to understand.
- Reduces unnecessary searches.
- Guarantees a valid solution.
- Efficient for constraint-based problems.

Conclusion

The 8-Queens Problem is a classic Constraint Satisfaction Problem in Artificial Intelligence. It is efficiently solved using the Backtracking Search Algorithm, which places queens one by one and removes them whenever a conflict occurs. This method ensures that all eight queens are placed safely without attacking each other and demonstrates the effectiveness of Depth First Search (DFS) in solving AI search problems.

Question 4: OLA Cab Booking System as a Goal-Based Agent

Aim

To design an OLA Cab Booking System as a Goal-Based Intelligent Agent and explain its working with algorithm and pseudocode.

Problem Statement

An OLA Cab Booking System is an intelligent application that helps users book taxis to travel from one location to another.

The system acts as a Goal-Based Agent, where its goal is to find the most suitable cab for the customer based on the pickup location, destination, driver availability, distance, and estimated fare.

Goal-Based Agent

A Goal-Based Agent chooses actions that help achieve a specific goal.

- Find the nearest available cab.
- Book the cab successfully.
- Reach the destination safely.
- Complete payment and trip.

Components of the OLA Cab Booking Agent

Percepts (Inputs)

- User pickup location
- Destination
- GPS location

- Available drivers
- Traffic conditions
- Estimated fare
- Customer booking request

Actions

- Search nearby cabs
- Calculate fare
- Assign nearest driver
- Send booking confirmation
- Track live location
- Complete payment
- End the trip

Environment

The agent operates in:

- Road network
- GPS system
- Customer mobile app
- Driver mobile app
- Traffic conditions
- Payment gateway

Performance Measures

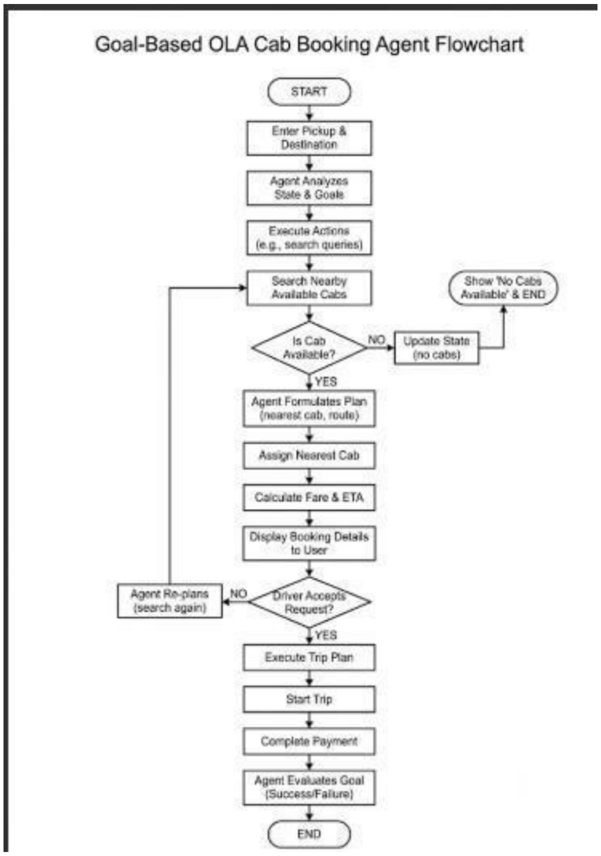
The agent should:

- Minimize waiting time

- Find the shortest route
- Reduce travel time
- Provide accurate fare
- Ensure customer satisfaction

Working Steps

1. The customer opens the OLA application.
2. The user enters the pickup and destination locations.
3. The system checks for nearby available drivers.
4. The nearest suitable driver is selected.
5. The estimated fare and arrival time are displayed.
6. The booking request is sent to the driver.
7. After driver acceptance, trip details are confirmed.
8. The customer tracks the cab in real time.
9. The driver reaches the destination.
10. Payment is completed and the trip ends.



Algorithm

1. Start the booking process.
2. Accept the pickup and destination locations.
3. Search for nearby available drivers.
4. If no driver is available, display a message and stop.
5. Assign the nearest available driver.
6. Calculate the estimated fare and travel time.
7. Send the booking request to the driver.
8. If the driver accepts, start the trip.
9. Track the ride until the destination is reached.
10. Complete payment and end the trip.

Pseudocode

```
BEGIN
Input PickupLocation
Input Destination
Search AvailableDrivers
IF AvailableDrivers = NULL THEN
   Display "No Cab Available"
   STOP
ENDIF
Assign NearestDriver
Calculate Fare
Send Booking Request
IF Driver Accepts THEN
   Start Trip
   Track Ride
   Complete Payment
ELSE
   Assign Next Driver
ENDIF
Display "Trip Completed"
END
```

Advantages

- Reduces customer waiting time.
- Finds the nearest available driver.
- Provides real-time tracking.
- Calculates fare automatically.
- Improves customer convenience.
- Supports secure online payment.

Applications

- OLA
- Uber
- Rapido
- Taxi booking services
- Ride-sharing platforms

Conclusion

The OLA Cab Booking System is an example of a Goal-Based Intelligent Agent because it selects actions based on the goal of successfully booking a cab and completing the trip. It continuously evaluates driver availability, traffic conditions, and route information to provide efficient and reliable transportation. This approach improves customer satisfaction by reducing waiting time, optimizing routes, and ensuring safe and convenient travel.

Question 5: Uniform Cost Search (UCS) – Logistics Delivery Network

Aim

To determine the least-cost path from Warehouse S (Source) to Warehouse G (Goal) using the Uniform Cost Search (UCS) algorithm.

Problem Statement

A logistics company transports packages between warehouses. Each road has a travel cost (fuel + time). The objective is to find the minimum-cost path from **S** to G using the Uniform Cost Search (UCS) algorithm.

Given Weighted Graph

Edges and Costs:

Edge	Cost
S → A	1
S → G	12
A → B	3
A → C	1
B → D	3
C → D	1
C → G	2

Edge	Cost
D → G	3

Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the lowest cumulative path cost first. It guarantees the optimal solution when all edge costs are positive.

UCS Working Steps

Step 1

Start at **S**.

Frontier: `(S, 0)`

Step 2

Expand **S**.

Possible paths:

- S → A = **1**
- S → G = **12**

Choose **A** (lowest cost).

Step 3

Expand **A**.

Possible paths:

- S → A → B = **4**
- S → A → C = **2**

Choose **C** (lowest cost).

Step 4

Expand **C**.

Possible paths:

- S → A → C → D = **3**
- S → A → C → G = **4**

Choose **D** (cost = 3).

Step 5

Expand **D**.

New path:

- S → A → C → D → G = **6**

The frontier now contains:

- G = **4**
- B = **4**
- G = **6**
- G = **12**

Choose **G (cost = 4)**.

Goal reached.

UCS Expansion Table

Step	Expanded Node	Path Cost
1	S	0
2	A	1
3	C	2
4	D	3
5	G	4

Least-Cost Path

```
S → A → C → G
```

Total Cost

```
1 + 1 + 2 = 4
```

Algorithm

1. Start from the source node **S**.
2. Insert **S** into the priority queue with cost **0**.
3. Remove the node with the lowest cumulative cost.
4. Expand the selected node and calculate the cost of its neighbors.
5. Insert unexplored neighbors into the priority queue.
6. Repeat until the goal node **G** is removed from the queue.
7. Return the path with the minimum total cost.

Pseudocode

```
BEGIN

Insert Source S into Priority Queue with cost 0

WHILE Priority Queue is not empty

   Remove node with minimum cost

   IF node = Goal THEN
      Print Path and Cost
      STOP
   ENDIF

   FOR each neighboring node

      NewCost = CurrentCost + EdgeCost

      Insert neighbor with NewCost

   END  FOR

END WHILE

END
```

Advantages

- Finds the optimal (minimum-cost) path.
- Suitable for weighted graphs.
- Always expands the least-cost node first.
- Guarantees the shortest path when edge costs are non-negative.

Applications

- GPS navigation systems
- Logistics and delivery routing
- Robot path planning
- Network routing
- Supply chain optimization

Conclusion

The Uniform Cost Search (UCS) algorithm successfully finds the minimum-cost path from S **to** G**.**

Final Answer

- **Optimal Path: S → A → C → G**
- **Minimum Cost: 4**

This is the least-cost route for transporting the package through the warehouse network.